



<Dead Harvest> Especificación de Requisitos de Software (ERS)

Versión: 0200 (Correcciones)

Fecha: 29/07/2026

Versión del Producto: 1.0 - MVP

Equipo: Iron Pixels

GitHub Project: <https://github.com/Crissssss2401/iron-pixels-is2.git>

Queda prohibido cualquier tipo de explotación y, en particular, la reproducción, distribución, comunicación pública y/o transformación, total o parcial, por cualquier medio, de este documento sin el previo consentimiento expreso.

HOJA DE CONTROL

Campo	Valor
Organismo	Universidad / Curso Ingeniería de Software II
Proyecto	Dead Harvest - Videojuego 2D Supervivencia Cenital
Entregable	Especificación de Requisitos (ERS COMPLETA v2)
Autor	Equipo Iron Pixels - Marcos Sulugüí Pich, Cristian Sahon, Nahum Albeño, Gilmar Maldonado, Wendy Chumil, Hefer Cotúc
Versión/Edición	0200 - Correcciones
Fecha Versión	28/07/2026
Aprobado por	Ing. Curso IS II / Scrum Master Cristian Sahon
Fecha Aprobación	29/07/2026
Nº Total Páginas	22

REGISTRO DE CAMBIOS

Versión	Causa del Cambio	Responsable del Cambio	Fecha del Cambio
0100	Versión inicial - Creación ERS parcial y backlog	Iron Pixels	27/07/2026
0101	Adaptación completa a plantilla oficial y trazabilidad	Marcos Sulugüí	28/07/2026
0200	CORRECCIÓN ALINEADA A GITHUB: RF-03 (#23) y RF-04 (#24) reclasificados de columna RNF a RF. Alineación Backlog 14 HU con Issues #1-#25. Trazabilidad 100%	Equipo Iron Pixels	29/07/2026

CONTROL DE DISTRIBUCIÓN

Nombre y Apellidos	Rol
Cristian Sahn	Líder del equipo / Scrum Master
Marcos Sulugüí	Responsable documentación / Programador
Nahum Albeño	Responsable herramientas / Programador
Gilmar Maldonado	Responsable comunicación / Programador
Wendy Chumil	Responsable diseño UI/UX / Programadora
Hefer Cotúc	Responsable QA / Product Owner
Docente IS II	Revisión

ÍNDICE

1 INTRODUCCIÓN

1.1 Alcance

1.2 Objetivos

2 INFORMACIÓN DEL DOMINIO DEL PROBLEMA

2.1 Introducción al Dominio del Problema

2.2 Glosario de Términos

3 DESCRIPCIÓN DE LA SITUACIÓN ACTUAL

3.1 Pros y Contras

3.2 Modelos de Procesos Actuales

3.3 Entorno Tecnológico Actual

4 NECESIDADES DE NEGOCIO

4.1 Objetivos de Negocio

4.2 Modelos de Procesos a Implantar

5 DESCRIPCIÓN DE LOS SUBSISTEMAS

6 CATÁLOGO DE REQUISITOS

6.1 Requisitos Generales

6.2 Casos de Uso

6.3 Requisitos Funcionales

6.4 Requisitos No Funcionales

6.5 Restricciones Técnicas

6.6 Requisitos de Integración

6.7 Trazabilidad y Backlog Final

7 ANEXOS

7.1 Anexo A: Actas de Reuniones ACTUALIZADO

1 INTRODUCCIÓN

Este documento presenta la Especificación de Requisitos de Software (ERS) COMPLETA v2 correspondiente al proyecto Dead Harvest, un videojuego de acción y supervivencia en 2D con perspectiva cenital desarrollado como proyecto integrador del curso de Ingeniería de Software II por el equipo Iron Pixels. Su propósito es definir de manera clara y organizada los requerimientos funcionales y no funcionales que servirán como base para el diseño, desarrollo y validación del sistema. La necesidad surge de la asignatura de crear un MVP jugable que demuestre aplicación de metodologías ágiles, ingeniería de requisitos y desarrollo en Unity.

1.1 Alcance

La primera versión de Dead Harvest contempla el desarrollo de un videojuego en Unity 2D con perspectiva cenital, orientado a partidas de aproximadamente 20 minutos. El jugador controlará a un único personaje que deberá sobrevivir a oleadas de zombis usando solo armas cuerpo a cuerpo, administrando estamina y botiquines. Incluye: movimiento 8 direcciones, 4 armas (espada, hacha, mazo, lanza), 4 tipos de zombis (Caminante, Corredor, Acorazado, Explosivo), sistema de oleadas con incremento de dificultad, IA por FSM, HUD, menú principal, pantalla de resultados y jefe final. Fuera de alcance v1.0: Multijugador, guardado persistente, microtransacciones, personalización de skins, IA con Machine Learning, niveles procedurales, contenido descargable.

1.2 Objetivos

- OBJ-01: Desarrollar un MVP jugable de supervivencia 2D con combate cuerpo a cuerpo fluido.
- OBJ-02: Implementar IA diferenciada por tipo de enemigo que genere tensión constante.
- OBJ-03: Implementar sistema de oleadas progresivo y balanceado.
- OBJ-04: Ofrecer interfaz clara y controles responsivos (<100ms).
- OBJ-05: Aplicar principios de Ingeniería de Software: ERS, Backlog priorizado MoSCoW, trazabilidad y validación 100% alineada a GitHub Projects.
- OBJ-06: Entregar build jugable a 60 FPS estable en PC.

2 INFORMACIÓN DEL DOMINIO DEL PROBLEMA

2.1 Introducción al Dominio del Problema

Dead Harvest pertenece al dominio de videojuegos de supervivencia acción top-down. Conceptos clave del dominio: Superviviente controla avatar en arena cerrada, horda de infectados generados por oleadas, gestión de recursos limitados (vida, estamina), combate melee basado en hitbox y cooldown, progresión por rondas y meta de escapar de zona de cuarentena. El jugador no dispara, depende de posicionamiento, timing y administración de estamina para esquivar. El dominio incluye patrones como FSM para IA, object pooling para enemigos, y HUD diegético minimalista.

2.2 Glosario de Términos

- Acorazado: Zombi con alta vida y armadura que ignora parcialmente daño de armas ligeras.
- Botiquín: Objeto recolectable que restaura 25% de vida.
- Caminante: Zombie básico lento, patrulla aleatoria hasta detectar al jugador.
- Corredor: Zombi rápido con poca vida.
- Dash / Esquiva: Movimiento rápido que consume estamina e otorga invulnerabilidad breve.
- Estamina: Recurso que se consume al atacar y esquivar, se regenera en reposo.
- Explosivo: Zombi que explota al morir causando daño en área.
- FSM: Finite State Machine - Máquina de estados finitos para IA (Patrullar, Perseguir, Atacar).
- Hitbox: Área de colisión invisible del arma que detecta impacto.
- HUD: Head-Up Display - Interfaz en pantalla con vida, estamina, oleada, arma.
- Oleada: Ronda de enemigos con cantidad y dificultad creciente.
- Pooling: Técnica de reutilización de objetos para optimizar el rendimiento.
- MoSCoW: Técnica de priorización (Must, Should, Could, Won't).
- RF: Requisito Funcional - GitHub columna Backlog RF.
- RNF: Requisito No Funcional - GitHub columna Backlog RNF.

3 DESCRIPCIÓN DE LA SITUACIÓN ACTUAL

3.1 Pros y Contras de la Situación Actual

3.1.1 Fortalezas

ID: F-001 - Conocimiento en Unity 2D básico del equipo. Versión: 1.0 (27/07/2026) Descripción: El equipo ya ha trabajado con Unity, C# y GitHub, lo que facilita prototipado rápido. Comentarios: Mantener uso de GitFlow y reuniones semanales.

3.1.2 Debilidades

ID: D-001 - Sin juego base previo ni assets definitivos. Versión: 1.0 (27/07/2026) Descripción: No existe código reutilizable de un proyecto similar. Todo debe crearse desde cero. Comentarios: Mitigar con assets temporales y uso de Unity Asset Store gratuitos.

3.2 Modelos de Procesos de Negocio Actuales

ID: AN-001 Jugador / AN-002 Equipo de Desarrollo. Proceso actual es inexistente, se crea nuevo. Antes del 27/07/2026 no existía proceso formal, solo ideas sueltas (PN-001 Desarrollo Ad-hoc previo).

3.3 Entorno Tecnológico Actual

Hardware: PCs de desarrollo Windows 11, 16GB RAM, GPU dedicada. Hardware objetivo: PC gama media con teclado y mouse. Software: Unity 2022.3 LTS, Visual Studio 2022, GitHub (<https://github.com/Crissssss2401/iron-pixels-is2.git>), Project Board con columnas Backlog RF, Backlog RNF, Backlog, Sprint 1, Roadmap. No hay backend, arquitectura cliente local.

4 NECESIDADES DE NEGOCIO

4.1 Objetivos de Negocio

ID	Descripción	Importancia	Prioridad	Dependencias
ON-01 Supervivencia Desafiante	El sistema deberá ofrecer partidas desafiantes de 15-20 min con dificultad progresiva que retenga al jugador.	Alta	Alta	-
ON-02 Combate Satisfactorio	El sistema deberá ofrecer feedback inmediato de combate (animación + sonido + partículas) en <100ms.	Alta	Alta	ON-01
ON-03 Aplicar Ingeniería de Software	El sistema deberá ser trazable desde objetivos hasta código mediante ERS y Backlog MoSCoW 100% alineado a GitHub.	Alta	Alta	-
ON-04 Entregable Académico	Cumplir con plantilla ERS 29 páginas y 8+ historias de usuario para evaluación IS II.	Alta	Alta	ON-03

4.2 Modelos de Procesos de Negocio a Implantar

ANI-01 Jugador - Rol principal que inicia partida, sobrevive oleadas, recoge botiquines y decide mejoras.
Dependencias: AN-001. PNI-01 Flujo de Partida Dead Harvest: Jugador inicia en Menú Principal -> Nueva Partida -> Aparece en Arena -> Oleada N inicia -> Jugador combate y gestiona estamina -> Recoge botiquines -> Oleada superada -> Elige mejora -> Siguiente oleada (dificultad++) -> Jefe Final -> Pantalla Resultados -> Jugar de nuevo / Menú.

5 DESCRIPCIÓN DE LOS SUBSISTEMAS DEL SISTEMA A DESARROLLAR

- SUB-01 Gameplay Core - Movimiento 8 direcciones, combate cuerpo a cuerpo, cambio de arma Q/E, estamina, dash, vida, recolección de botiquines.
- SUB-02 Inteligencia Artificial - FSM de enemigos (Patrullar, Perseguir, Atacar), percepción, tipos Caminante/Corredor/Acorazado/Explosivo, pooling. CORREGIDO: RF-03 #23.
- SUB-03 Sistema de Oleadas - Spawner, conteo, incremento de dificultad, pausa entre oleadas, jefe final. CORREGIDO: RF-04 #24.
- SUB-04 UI / UX - HUD (vida, estamina, oleada, arma), Menú Principal, Pausa, Pantalla de Resultados.
- SUB-05 Audio y Efectos - SFX de impacto, música de tensión, partículas de golpe.

6 CATÁLOGO DE REQUISITOS DEL SISTEMA A DESARROLLAR

6.1 Requisitos Generales del Sistema

ID	Nombre	Descripción	Dependencias	Importancia	Prioridad
RG-01	Sistema de Combate Cuerpo a Cuerpo	El sistema deberá proporcionar combate melee con 4 armas diferenciadas.	ON-02	Alta	Alta
RG-02	Sistema de IA de Enemigos	El sistema deberá implementar IA con comportamientos diferenciados por tipo. (RF-03 #23)	ON-01	Alta	Alta
RG-03	Sistema de Oleadas y Progresión	El sistema deberá gestionar oleadas con dificultad incremental. (RF-04 #24)	ON-01	Alta	Alta
RG-04	Sistema de Interfaz y Experiencia	El sistema deberá mostrar HUD, menú principal y resultados de forma clara.	ON-01	Media	Alta

6.2 Casos de Uso del Sistema

Diagrama UML sugerido: Actor Jugador conectado a CU-01 Atacar, CU-02 Cambiar Arma, CU-03 Esquivar, CU-04 Recoger Botiquín, CU-05 Iniciar Partida, CU-06 Ver Resultados. Actor Sistema Oleadas conectado a CU-07 Generar Oleada. Incluir <<include>> entre CU-01 y CU-03 consumo estamina.

ACT-01 Jugador - Usuario final que controla al superviviente, inicia partida y ejecuta acciones de combate. ACT-02 Sistema de Juego - Actor sistema que gestiona oleadas, IA y reglas de negocio.

CU-01 Atacar Cuerpo a Cuerpo

Dependencias: RG-01, RF-01 #18 | Precondición: Jugador en partida, estamina >0, arma equipada | Secuencia: 1. Jugador presiona clic izquierdo 2. Sistema verifica estamina 3. Sistema ejecuta animación 4. Detecta colisión hitbox 5. Aplica daño y feedback | Postcondición: Enemigo con vida reducida, estamina reducida | Excepciones: Si estamina=0, ataque reducido

CU-02 Cambiar Arma

Dependencias: RG-01, RF-02 #19 | Precondición: En partida, al menos 2 armas | Secuencia: 1. Q/E 2. Cambia arma 3. Actualiza stats y HUD | Postcondición: Arma equipada actualizada | Excepciones: -

CU-03 IA Enemiga (Perseguir)

Dependencias: RG-02, RF-03 #23 CORREGIDO | Precondición: Enemigo activo en arena | Secuencia: 1. Patrulla 2. Detecta jugador en radio 3. Persigue 4. Ataca si en rango | Postcondición: Enemigo en estado Persecución/Ataque | Excepciones: Pierde visión -> vuelve a patrullar

CU-04 Sistema Oleadas

Dependencias: RG-03, RF-04 #24 CORREGIDO | Precondición: Oleada anterior superada | Secuencia: 1. Sistema calcula N+1 enemigos 2. Spawner genera por tipo 3. Inicia oleada | Postcondición: Oleada N activa | Excepciones: Si jugador muere, fin partida

6.3 Requisitos Funcionales del Sistema

6.3.1 Requisitos de Información

RI-01 Información del Jugador: vida actual, máxima, estamina actual, máxima, arma equipada, tiempo sobrevivido, enemigos eliminados. RI-02 Información del Enemigo: Tipo, vida, daño, velocidad, radio percepción, estado FSM. RI-03 Información de Oleada: Número de oleada, enemigos por tipo, tiempo de aparición.

6.3.2 Requisitos de Reglas de Negocio

- RN-01: La estamina nunca podrá ser negativa y no podrá superar su máximo.
- RN-02: Si vida ≤ 0 , partida termina en derrota.
- RN-03: Si estamina $= 0$, velocidad de ataque y movimiento se reduce 50% hasta recuperar 20%.
- RN-04: Botiquín no puede curar por encima de vida máxima.
- RN-05: Número de enemigos en oleada $N+1 > N$.

6.3.3 Requisitos de Conducta del Sistema

ID GitHub	Tipo Correcto	Nombre	Estado	Descripción Completa RU/RS	Dependencias
RF-01 #18	RF	Ataque cuerpo a cuerpo	OK en Backlog RF	El sistema deberá ejecutar ataque melee con animación, hitbox y daño según arma equipada. RU: Como jugador, quiero atacar cuerpo a cuerpo. RS1: Animación al clic izquierdo. RS2: Colisión hitbox. RS3: Daño arma. RS4: VFX/SFX.	RG-01, CU-01

RF-02 #19	RF	Cambio de armamento	OK en Backlog RF	El sistema deberá permitir cambiar arma con Q/E y actualizar stats inmediatamente. RU: Cambiar armas. RS1: Q/E. RS2: Reflejo inmediato. RS3: 4 armas: espada, hacha, mazo, lanza.	RG-01, CU-02
RF-03 #23	RF CORREGID O	Inteligencia Artificial De Los Enemigos	CORREGID O - Mover de RNF a RF	El sistema deberá implementar FSM: patrullar, perseguir, atacar. RU: Zombis creíbles. RS1: FSM. RS2: Perseguir si entra en radio. RS3: Atacar si en rango ataque. Acción correctiva: Se encontraba en columna Backlog RNF por error, ahora en Backlog RF.	RG-02, CU-03

RF-04 #24	RF CORREGID O	Sistema De Oleadas	CORREGID O - Mover de RNF a RF	El sistema deberá coordinar aparición y cantidad por ronda con incremento gradual. RU: Oleadas crecientes. RS1: Coordinar cuándo y cuántos. RS2: Incrementar dificultad. Acción correctiva: Se encontraba en columna Backlog RNF por error, ahora en Backlog RF.	RG-03, CU-04
RF-05 #11	RF	HUD (Head- Up Display)	OK en Backlog RF	El sistema deberá mostrar barra vida, estamina, oleada y arma, actualizando en tiempo real. RU: Ver estado sin interrumpir. RS1: Mostrar vida, estamina, oleada, arma. RS2: Tiempo real.	RG-04
RF-06 #12	RF	Menú Principal	OK en Backlog RF	El sistema deberá presentar menú al iniciar con Nueva Partida, Opciones, Salir. RU: Acceder a menú. RS1: Opciones. RS2: Navegación teclado/mouse.	RG-04

RF-07 #13	RF	Pantalla de Resultados	OK en Backlog RF	Mostrar resultado, tiempo, enemigos eliminados, oleada alcanzada. RU: Ver resumen. RS1: Mostrar victoria/derrota, tiempo, kills, oleada. RS2: Jugar nuevamente / Menú.	RG-04
RF-08 #20	RF	Sistema de estamina	OK en Backlog RF	El sistema deberá reducir estamina al atacar/esquivar y regenerar en reposo, penalizar si llega a 0. RU: Gestión estratégica. RS1: Reducir al atacar/esquivar. RS2: Regenerar. RS3: Si 0, reducir velocidad.	RG-01

6.4 Requisitos No Funcionales del Sistema

ID GitHub	Tipo	Nombre	Estado	Descripción	Dependencias
-----------	------	--------	--------	-------------	--------------

RNF-01 #25	RNF	Rendimiento	OK en Backlog RNF	El sistema deberá soportar 50 zombis en pantalla sin bajar de 60 FPS. Métrica: FPS promedio >60 con 50 entidades. Importancia Alta. RU: Juego fluido incluso con muchos enemigos. RS: Soportar 50 simultáneos.	RG-01, RG-02, RG-03, ON-01
RNF-02 #14	RNF	Usabilidad	OK en Backlog RNF	90% jugadores deben iniciar partida e identificar vida, estamina y oleada en 60s sin ayuda. Importancia Alta. RU: Entender menú y HUD sin instrucciones. RS: 90% usuarios logran identificar en 60s.	RG-04, ON-02
RNF-03 #21	RNF	Tiempo de respuesta	OK en Backlog RNF	Procesar entrada jugador en <100ms. Importancia Crítica. RU: Controles inmediatos. RS: Entrada -> reflejo pantalla <100ms.	RG-01, ON-01

6.5 Restricciones Técnicas del Sistema

- RT-01: El sistema deberá desarrollarse en Unity 2022.3 LTS con C# . Importancia: Alta | Prioridad: Alta | Estado: Aprobado
- RT-02: Uso de Input System nuevo de Unity para compatibilidad teclado/mouse.
- RT-03: Control de versiones obligatorio en GitHub <https://github.com/Crissssss2401/iron-pixels-is2.git>
- RT-04: No usar assets de pago, solo gratuitos o propios por licencia académica.

6.6 Requisitos de Integración del Sistema

- RI-INT-01: El sistema deberá utilizar Unity Input System para manejo de entradas.
- RI-INT-02: El sistema deberá utilizar Unity UGUI para HUD y menús.
- RI-INT-03: Integración con Unity Animator para FSM de animaciones.

6.7 Información Sobre Trazabilidad y Backlog Final Priorizado MoSCoW

RG	RF (Corregido)	CU	HU / Backlog	ON / RT
RG-01	RF-01 #18, RF-02 #19, RF-08 #20	CU-01, CU-02	HU-01, HU-02, HU-07, HU-05	RT-01
RG-02	RF-03 #23 CORREGIDO	CU-03	HU-03, HU-08, HU-09	ON-01
RG-03	RF-04 #24 CORREGIDO, RF-08 #20	CU-04, CU-07	HU-04, HU-06, HU-10	ON-01

RG-04	RF-05 #11, RF-06 #12, RF-07 #13	CU-05, CU-06	HU-11, HU-12, HU-13	ON-02
ON-01	RNF-01 #25, RNF-03 #21		HU-10b Balanceo	
ON-02	RNF-02 #14			

Backlog Final Priorizado MoSCoW

Issue GitHub	Historia	Criterios Aceptación	MoSCoW	RF Relacionado
#1 HU-01 Movimiento	Como jugador, quiero moverme en 8 direcciones para explorar la arena	WASD mueve personaje, Sin atravesar muros	Must Have - Sprint 1	RF Base
#2 HU-02 Ataque	Como jugador, quiero atacar cuerpo a cuerpo para eliminar zombis	Clic ejecuta animación, Hitbox aplica daño	Must Have - Sprint 1	RF-01 #18
#3 HU-03 IA básica	Como jugador, quiero que zombis me persigan	Detecta en radio, Persigue y ataca	Must Have - Sprint 1	RF-03 #23
#4 HU-04 Oleadas	Como jugador, quiero oleadas que aparezcan	Spawner genera por ronda	Must Have - Sprint 1	RF-04 #24
#5 HU-05 HUD	Como jugador, quiero ver vida y estamina	HUD visible, Actualización tiempo real	Must Have - Sprint 1	RF-05 #11
#6 HU-06 Mejoras	Como jugador, quiero mejoras entre oleadas para ser más fuerte	Al terminar oleada muestra 3 mejoras, Selección aplica efecto	Should Have	RF-04 #24
#7 HU-07 Cambio armas	Como jugador, quiero cambiar armas con Q/E	Q/E cambia, Actualiza stats	Must Have	RF-02 #19
#8 HU-08 Zombis especiales	Como jugador, quiero 4 tipos de zombis	Caminante, Corredor, Acorazado, Explosivo con stats distintos	Should Have	RF-03 #23

#9 HU-09 Jefe final	Como jugador, quiero jefe final en última oleada	Aparece en oleada 10, Alta vida y daño	Could Have	RF-03/RF-04
#22 HU-10 Dificultad	Como jugador, quiero dificultad incremental por oleada	N+1 > N enemigos, Pausa breve	Must Have	RF-04 #24
#10 HU-10b Balanceo	Como PO, quiero balanceo de daño y estamina	Tabla balanceo documentada	Should Have	RNF-01/03
#15 HU-11 HUD visual	Como jugador, quiero HUD claro	Vida, estamina, oleada, arma visibles	Must Have	RF-05 #11
#16 HU-12 Menú	Como jugador, quiero menú principal	Nueva Partida, Opciones, Salir	Must Have	RF-06 #12
#17 HU-13 Resultados	Como jugador, quiero pantalla resultados	Tiempo, kills, oleada, Botones rejugar/menú	Must Have	RF-07 #13

7 ANEXOS

7.1 Anexo A: Actas de Reuniones

ACTA-001 - Reunión Inicial - Lunes 27/07/2026 - 09:00 PM - Google Meet Virtual

Link: Google Meet - Virtual

Asistentes: Cristian Sahon (Líder/Scrum Master), Marcos Sulugüí (Responsable Documentación / Programador), Nahum Albeño (Responsable Herramientas / Programador), Gilmar Maldonado (Responsable Comunicación / Programador), Wendy Chumil (Responsable UI/UX / Programadora), Hefer Cotúc (Responsable QA / Product Owner)

Agenda:

1. Creación de organización GitHub @Crisssss2401's Iron Pixels y Project Board con columnas Backlog RF, Backlog RNF, Backlog, Sprint 1, Roadmap.
2. Definición de 6 RF principales iniciales (RF-01,02,05,06,07,08) y 3 RNF (RNF-01,02,03). Se detecta posteriormente que RF-03 y RF-04 se agregaron por error en columna RNF en GitHub (#23 y #24).
3. Definición de MVP: Movimiento, ataque, IA básica, oleadas, HUD en Sprint 1 (Issues #1 a #5).
4. Asignación: HU-01 a chumilwen y Criss, HU-02 a chumilwen y GilFer, documentar ERS con RU y RS por cada RF.
5. Métodos de recolección: reuniones, entrevistas internas, análisis de juegos similares (Vampire Survivors, Dead Cells) y lluvia de ideas.
6. Acuerdos: Crear ERS parcial versión 0100, backlog inicial con MoSCoW, prototipo movimiento + ataque para 28/07.

Próxima reunión: 29/07/2026 revisión de GITHUB.